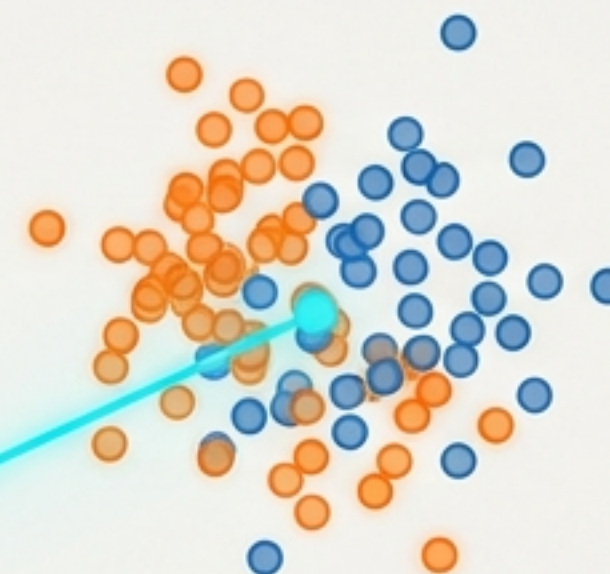


Visual Mastery with Seaborn

`hue="gender"`



A definitive guide to mapping multidimensional data into flat visual space using the grammar of graphics.

The Environment Architecture

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```



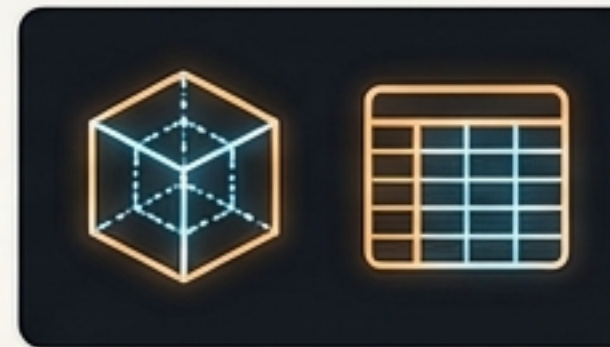
Layer 3: High-Level Interface

High-level declarative grammar for statistical graphics (seaborn, plotly).



Layer 2: The Render Engine

The foundational plotting engine powering Python (matplotlib).



Layer 1: The Foundation

Core mathematical and tabular data structures (numpy, pandas).

The Data Ingestion Pipeline



The pipeline requires two authorizations: mounting the external Google Drive volume to the Colab environment, and invoking Pandas to read the targeted file path into active memory as a DataFrame.

Anatomy of the Dataset

student DataFrame: 95 rows × 10 columns

Continuous / Numerical Variables

study_hours

test_score

week

attendance_rate

Categorical / Discrete Variables

student_id

class_level (Freshman, Sophomore, Junior, Senior)

subject (Math, Science, History, Language)

gender (Male, Female)

hostel (True, False)

Tshirt_size (10, 20, 30, 40)

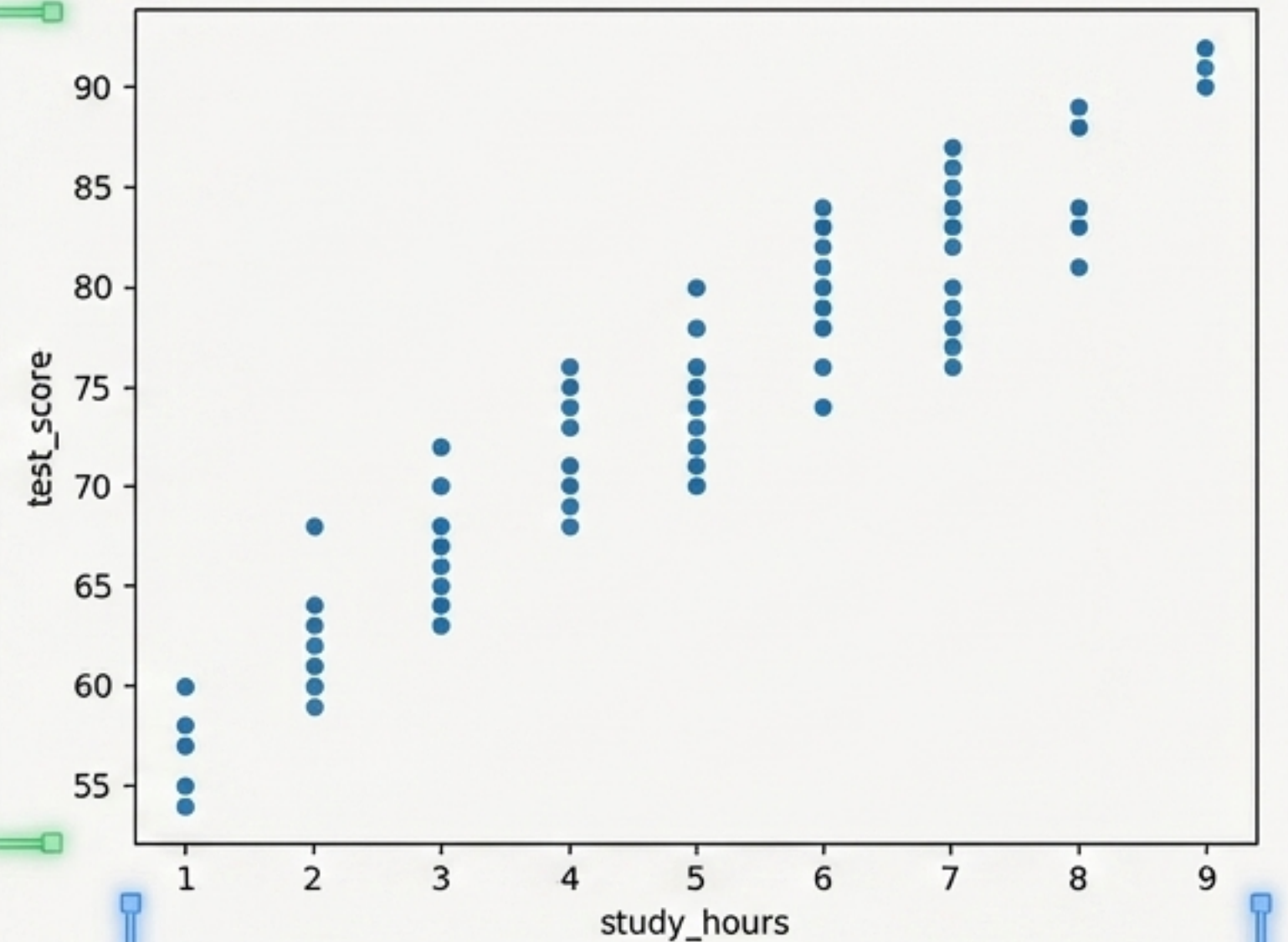
Context Note: The structural distinction between continuous axes and discrete categorical groupings dictates exactly how Seaborn maps values to rendering rendering shapes, scales, and colors.

Establishing Base Reality: 2D Space

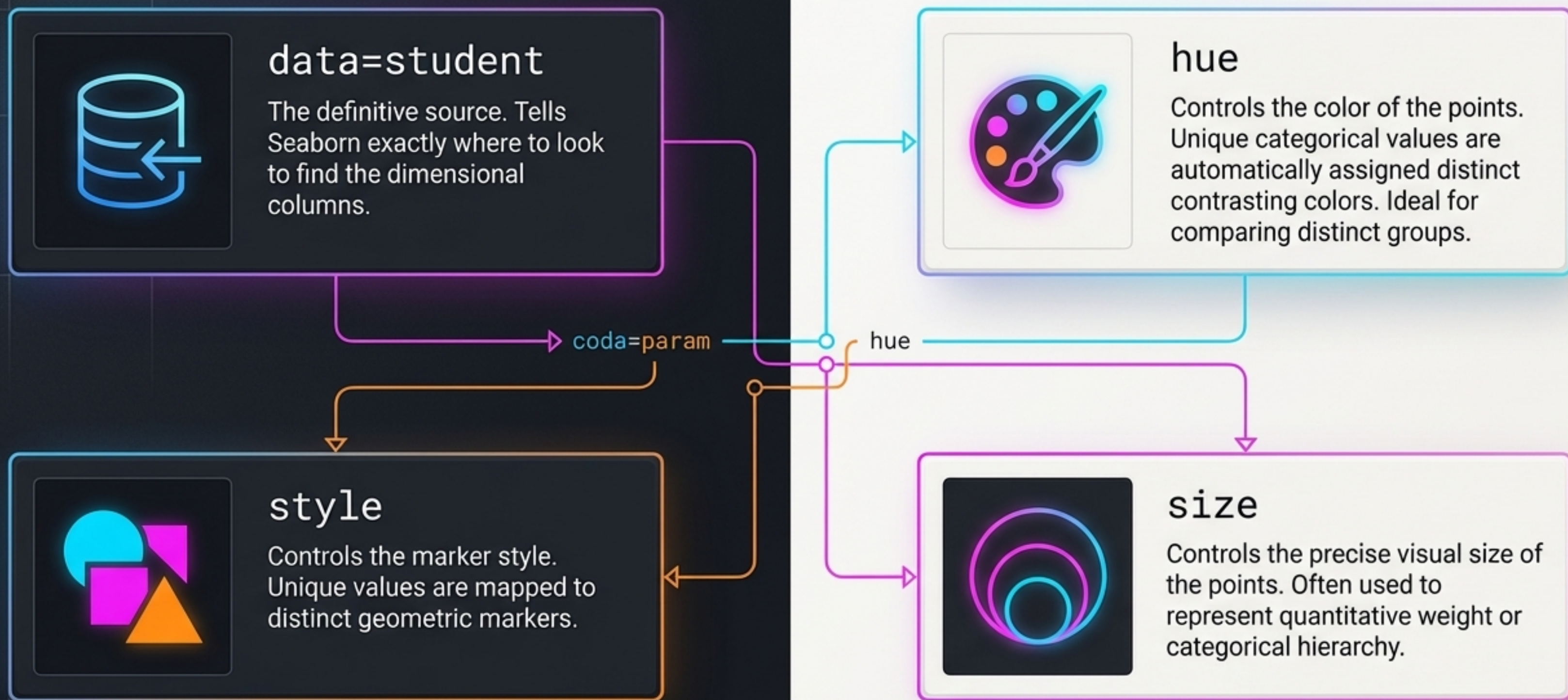


```
<Axes: xlabel='study_hours', ylabel='test_score'>  
sns.scatterplot(data=student, x="study_hours",  
                y="test_score")
```

This is the dataset base layer. Seaborn looks inside the student DataFrame and maps the continuous variables to simple Cartesian coordinates.



The Semantic Mapping Matrix

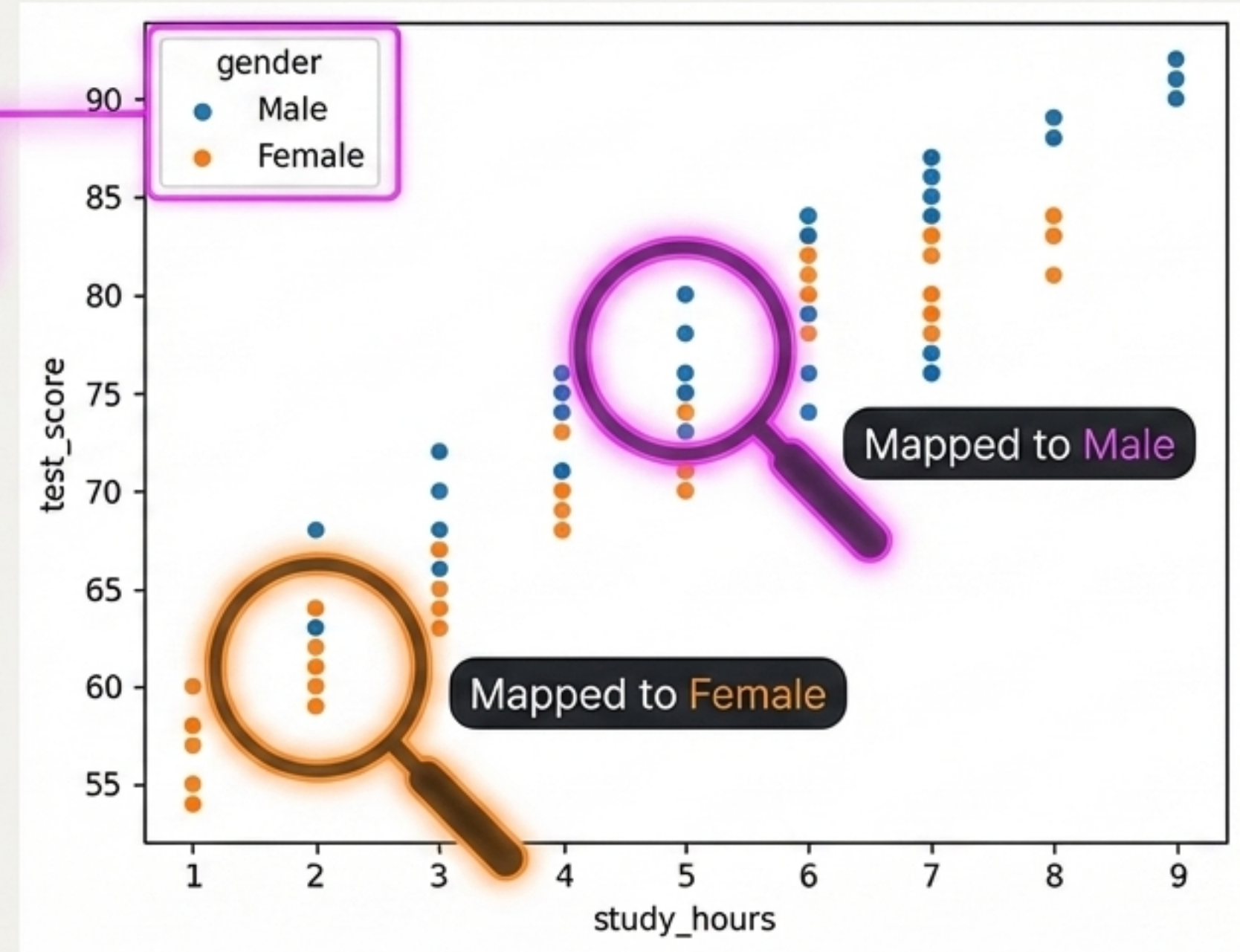


Layering Dimension 3: Color Encoding



```
# We can show it base on gender  
sns.scatterplot(data=student, x="study_hours",  
               y="test_score", hue="gender")
```

By injecting a single parameter, the 2D plane splits into distinct analytical groups without requiring multi-layered filtering loops in Pandas.

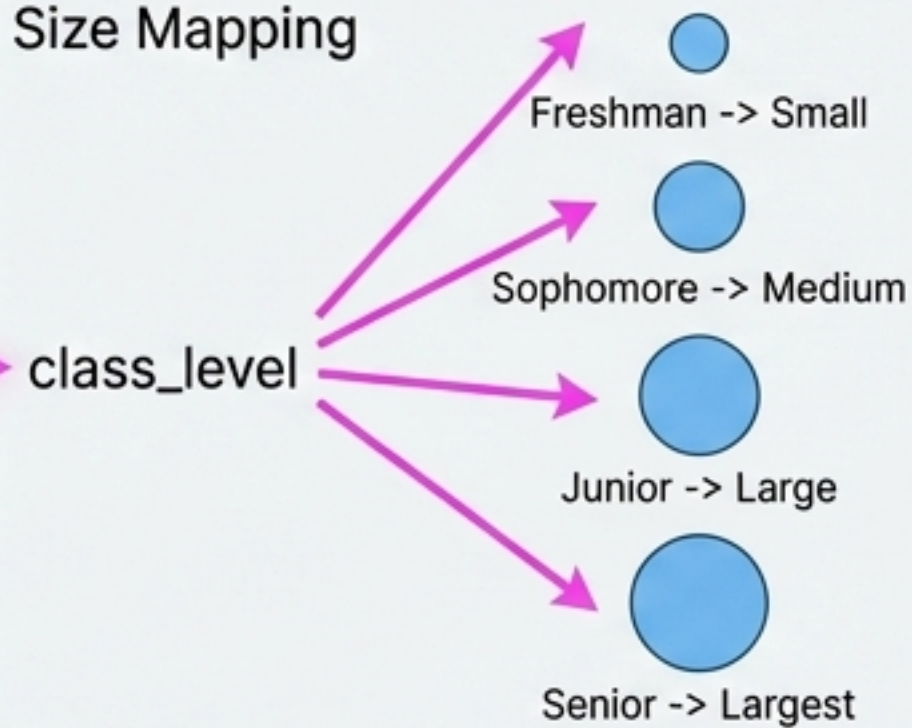


Encoding Shape and Scale

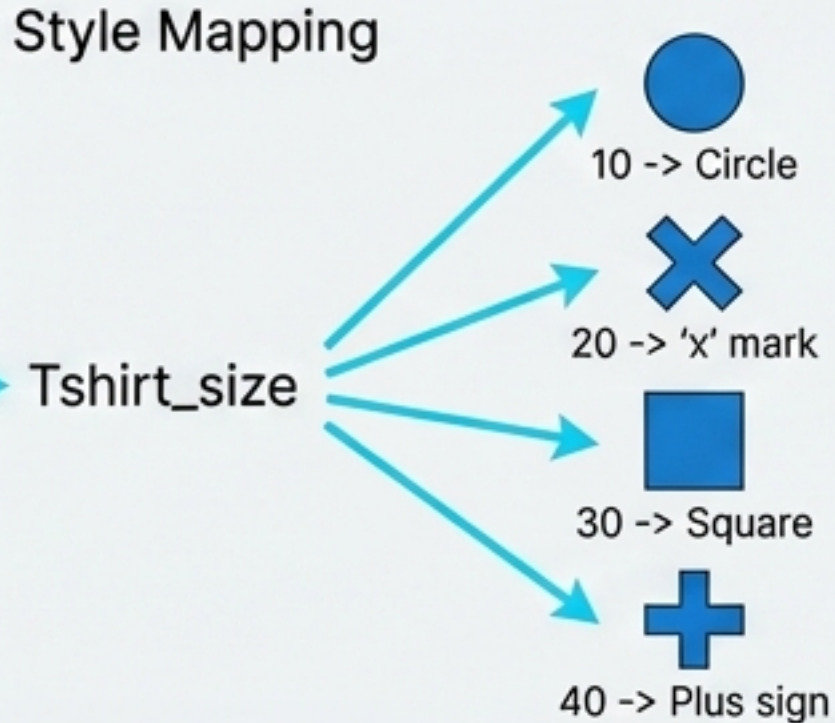


```
size="class_level", style="Tshirt_size"
```

Size Mapping

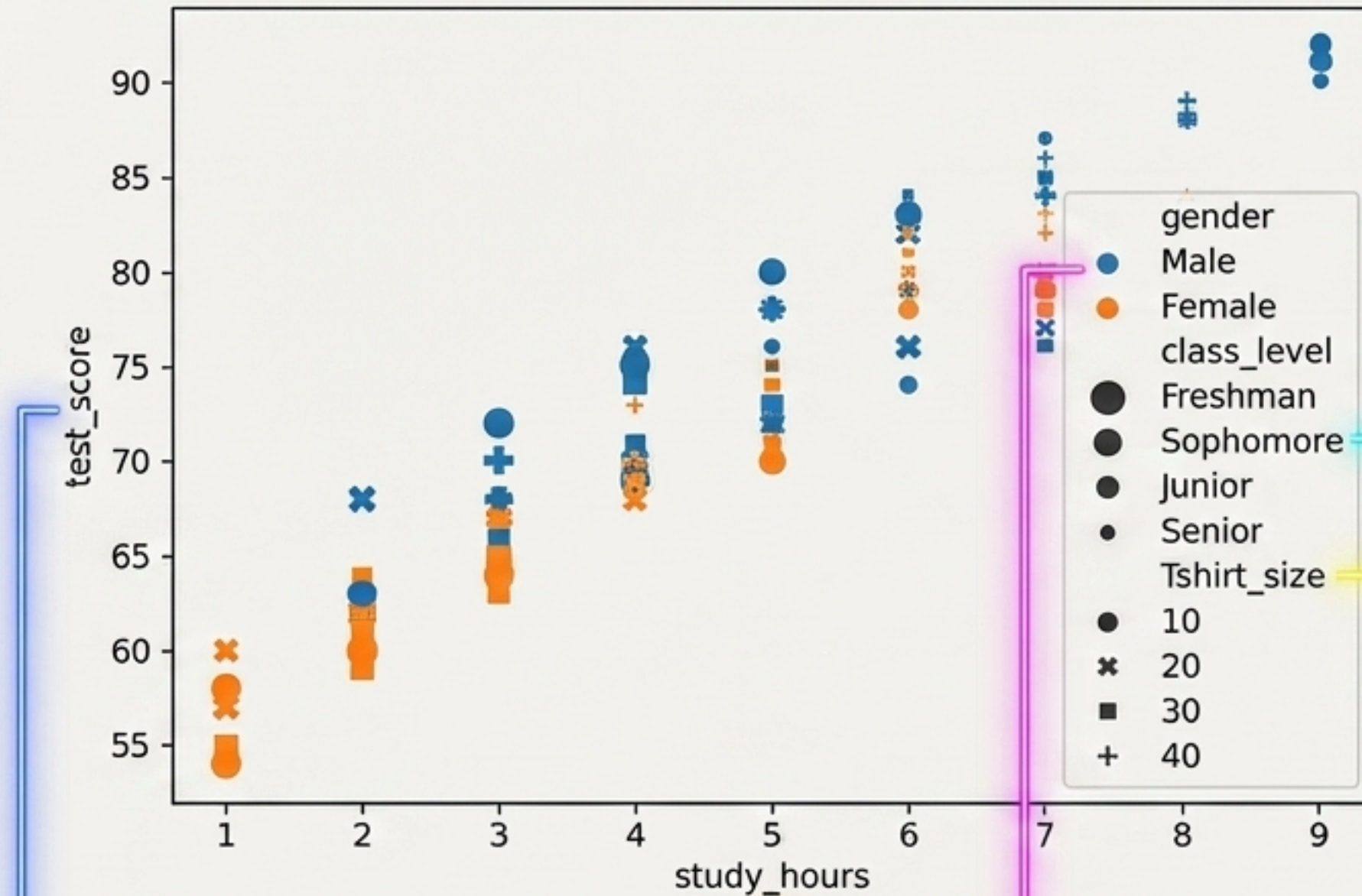


Style Mapping



In summary, this single line of code creates a complex plot that shows the relationship between study hours and test scores, while also encoding information about gender (by color), class level (by point size), and T-shirt size (by shape) for every single student.

The Multidimensional Masterpiece



```
sns.scatterplot(data=student, x="study_hours", y="test_score", hue="gender", size="class_level", style="Tshirt_size")
```

Alternative Architecture: Axes vs. Figure Functions

`sns.scatterplot` (Axes-Level)

```
<Axes: xlabel='study_hours', ...>
```



Axes-level functions draw onto a specific, singular set of matplotlib axes. They are discrete objects that fit inside an existing grid.

`sns.relplot` (Figure-Level)

```
<seaborn.axisgrid.FacetGrid at 0x7cd9e68f6c60>
```



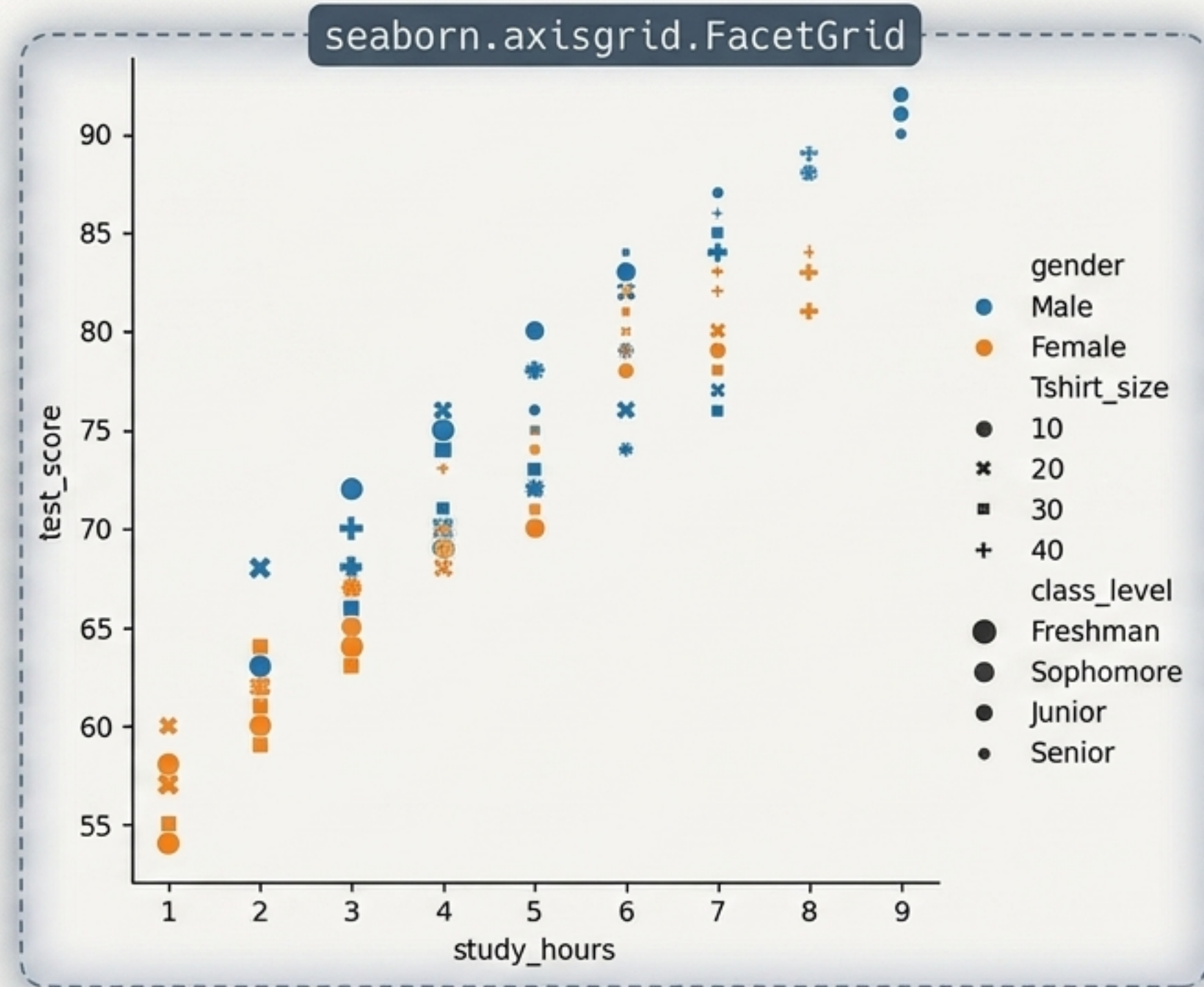
Figure-level functions manage their own entire Matplotlib figure. They instantiate a **FacetGrid**, allowing you to instantly generate multi-plot grids based on data subsets.

Identical Visuals, Divergent Structures



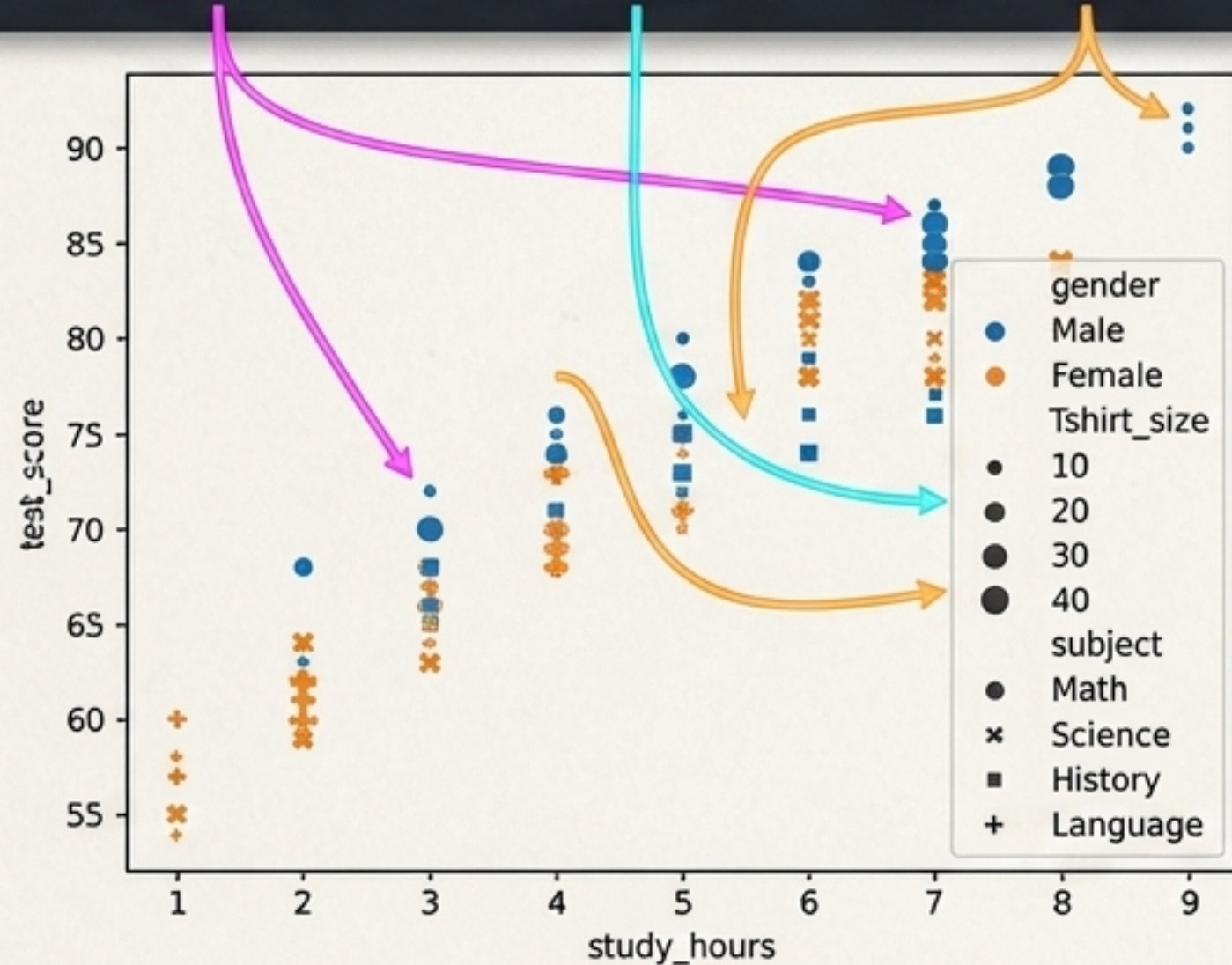
```
sns.relplot(kind="scatter",  
            data=student,  
            x="study_hours",  
            y="test_score",  
            hue="gender",  
            size="class_level",  
            style="Tshirt_size")
```

The visual mapping is identical, but relplot constructs a completely different object architecture, pulling the legend outside the axes space automatically.

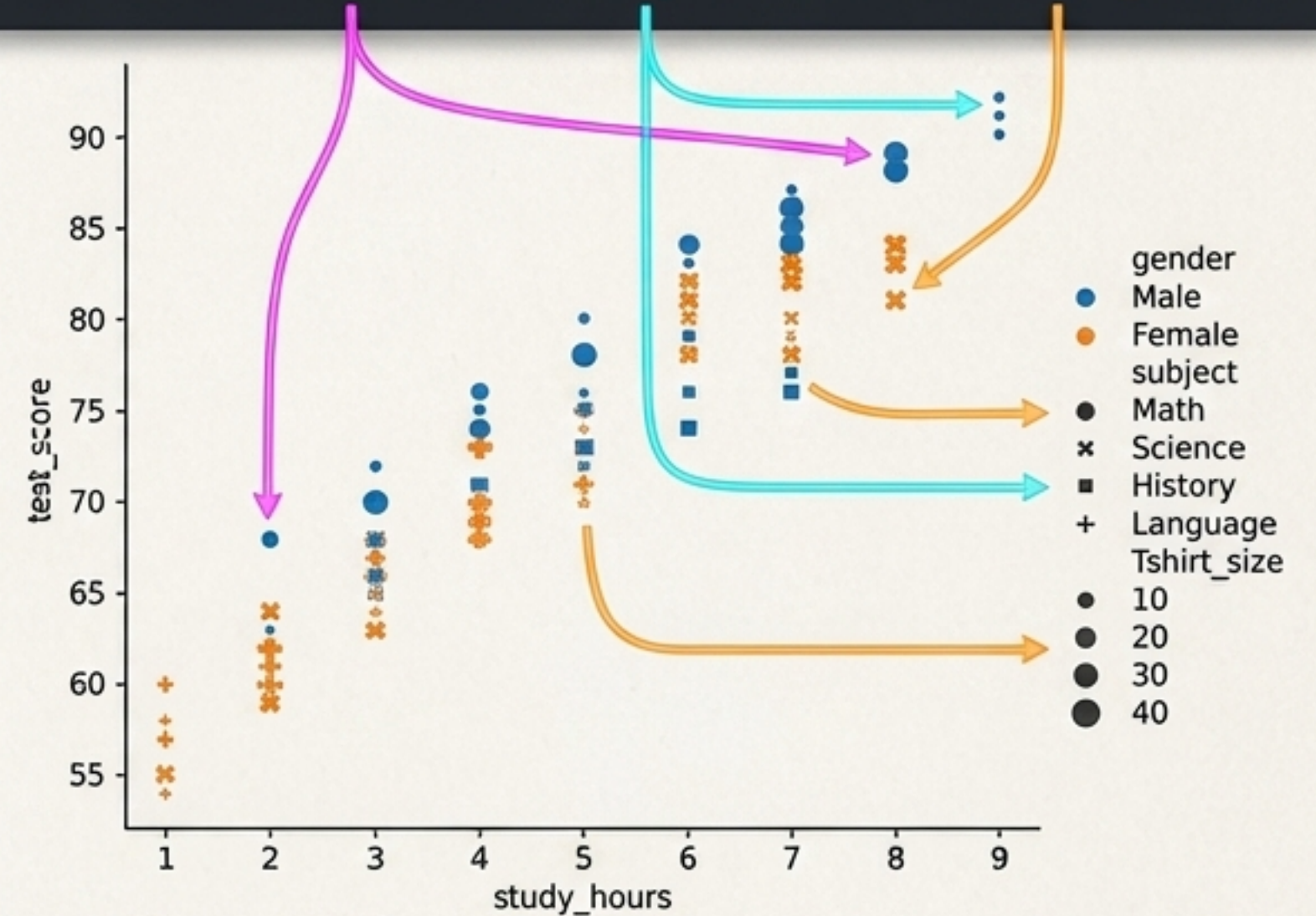


A Fluid Visual Vocabulary

```
hue='gender', size='Tshirt_size', style='subject')
```



```
sns.relplot(... style='subject', size='Tshirt_size')
```



The power of Seaborn lies not in drawing shapes, but in building a fluid grammatical vocabulary. By merely swapping column names mapped to 'style' and 'size', entirely new data narratives emerge instantly without rewriting complex plotting logic. The abstraction perfectly aligns with how a data scientist thinks.